

9 manage your data

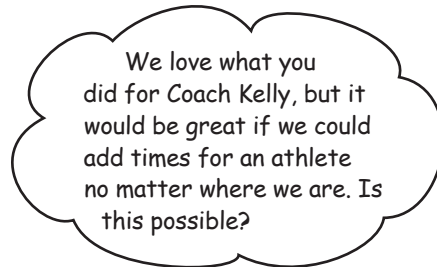
✧ *Handling input* ✧



The Web and your phone are not just great ways to display data.

They are also great tools to for accepting input from your users. Of course, once your webapp accepts data, it needs to put it somewhere, and the choices you make when deciding what and where this “somewhere” is are often the difference between a webapp that’s easy to grow and extend and one that isn’t. In this chapter, you’ll extend your webapp to accept data from the Web (via a browser or from an Android phone), as well as look at and enhance your back-end data-management services.

Your athlete times app has gone national



The *National Underage Athletics Committee (NUAC)* took one look at your Android app and realized it's just what they need...almost.

There are many ways to improve your webapp, but for now, let's concentrate on the committee's most pressing need: *adding a new time value to an existing athlete's data set*.

Adding new data to text files isn't going to work: there are just too many coaches around the country adding data. The committee wants something that's user friendly from any web browser or Android phone.

Can you help?

Use a form or dialog to accept input

Simply use the standard `<FORM>` and `<INPUT>` tags within your HTML web page to get input from your users...

...or if you are on your phone, a call to the `dialogGetInput()` function will do the trick.



On the Web, your user interacts with your web form and enters data. When she presses the **submit** button, the web browser gathers up all of the form's data and sends it to the web server *as part of the web request*.

On your Android phone, you can use the `dialogGetInput()` method to get input from the user, then mimic the behavior of the web form's **submit** button in code.

In fact, you've done this already: check out this line of code from your `coachapp.py` app, which sends the selected athlete name to your web server:

```
athlete = json.loads(send_to_server(web_server + get_data CGI, {'which_athlete': which_athlete}))
```

Here's where the data is included with the web request.

Create an HTML form template

Let's extend `yate.py` to support the creation of a HTML form. Take a look at this simple form, together with the HTML markup used to produce it.

The diagram illustrates the process of creating and using an HTML form. It consists of three main parts:

- Web Browser Window:** A window titled "form.html" showing a form with the text "Enter a timing value:" followed by a text input field and a "Send" button. A handwritten note with an arrow pointing to the button says: "Click the 'Send' button to submit the form's data to your web server."
- HTML Markup:** A code block showing the HTML for the form:


```
<form action="cgi-bin/process-time.py" method="POST">
Enter a timing value:
<input type="Text" name="TimeValue" size=40>
<br />
<input type="Submit" value="Send">
</form>
```

 A handwritten note with an arrow pointing to the `action` attribute says: "The name of the CGI script to send the form's data to."
- Python Code:** A code block showing the Python code to process the form data:


```
import cgi
form = cgi.FieldStorage()
timing_value = form["TimeValue"].value
```

 A handwritten note with an arrow pointing to the `form` variable says: "Get the data sent from the form as part of the web request." Another handwritten note with an arrow pointing to the `form["TimeValue"].value` expression says: "Access the value associated with the 'TimeValue' key from the form's data."

Arrows connect the "Send" button in the browser to the `action` attribute in the HTML, and the `form` variable in the Python code to the `form` tag in the HTML, indicating the flow of data from the user to the server and back to the application.

When your user clicks on the **Send** button, any data in the input area is sent to the web server as part of the web request.

On your web server, you can access the CGI data using the facilities provided by the standard library's `cgi` module:

The `cgi` module converts the data associated with the web request into a dictionary-like object that you can then query to extract what you need.



Exercise

Let's turn the HTML form from the previous page into a template within the `yate.py` module.

- 1 Start by creating a new template called `templates/form.html` that allows you to parameterize the form's CGI script name, method, input tags, and submit button text:

.....

.....

.....

- 2 With the template ready, write the code for two functions you intend to add to `yate.py`.

The first, called `create_inputs()`, takes a list of one or more strings and creates HTML `<INPUT>` tags for each string, similar to the one that accepts `TimeValue` on the previous page.

The second, called `do_form()`, uses the template from Part 1 of this exercise together with the `create_inputs()` function to generate a HTML form.

```
def create_inputs(inputs_list):
```

Given a list of `<INPUT>` tag names.

.....

.....

.....

.....

Return the generated tags to the caller.

```
    return(html_inputs)
```

The name of the CGI script and a list of `<INPUT>` tag names are required arguments.

```
def do_form(name, the_inputs, method="POST", text="Submit"):
```

.....

.....

.....

.....

The HTTP method and text to the "Submit" button have sensible default values.

Substitute the arguments and generated `<INPUT>` tags into the template to create the form.

```
    return(form.substitute(cgi_name=name, http_method=method,
                           list_of_inputs=inputs, submit_text=text))
```



Exercise Solution

You were to turn the HTML form into a template within the `yate.py` module.

- 1 You were to start by creating a new template called `templates/form.html` that allows you to parameterize the form's CGI script name, method, input tags, and submit button text.

The CGI script's name and associated HTTP method are parameterized.

```
<form action={cgi_name} method={http_method}>
```

```
Enter a timing value: {list_of_inputs}<br />
```

```
<input type="Submit" value={submit_text}></form>
```

The list of `<INPUT>` tags and the submit button's text is also parameterized.

2

With the template ready, you were to write the code for two functions to add to `yate.py`.

The first, called `create_inputs()`, takes a list of one or more strings and creates HTML `<INPUT>` tags for each string, similar to the one that accepts `TimeValue`.

The second, called `do_form()`, uses the template from Part 1 of this exercise together with the `create_inputs()` function to generate a HTML form:

```
def create_inputs(inputs_list):
```

```
    html_inputs = "
```

```
    for each_input in inputs_list:
```

```
        html_inputs = html_inputs + '<input type="Text" name="' +
```

```
        each_input + '" size=40>'
```

```
    return(html_inputs)
```

```
def do_form(name, the_inputs, method="POST", text="Submit"):
```

```
    with open('templates/form.html') as formf:
```

```
        form_text = formf.read()
```

```
        inputs = create_inputs(the_inputs)
```

```
        form = Template(form_text)
```

```
    return(form.substitute(cgi_name=name, http_method=method,
                          list_of_inputs=inputs, submit_text=text))
```

This "continuation" character lets you split a long line of code over multiple lines.

Take each name and create an `<INPUT>` tag.

Grab the template from your disk.

Create a template form.

Create the list of `<INPUT>` tags.



Test Drive

Here's the code to a CGI script called `cgi-bin/test-form.py`, which generates the HTML form from earlier. As you can see, there's nothing to it.

```
#!/usr/local/bin/python3

import yate

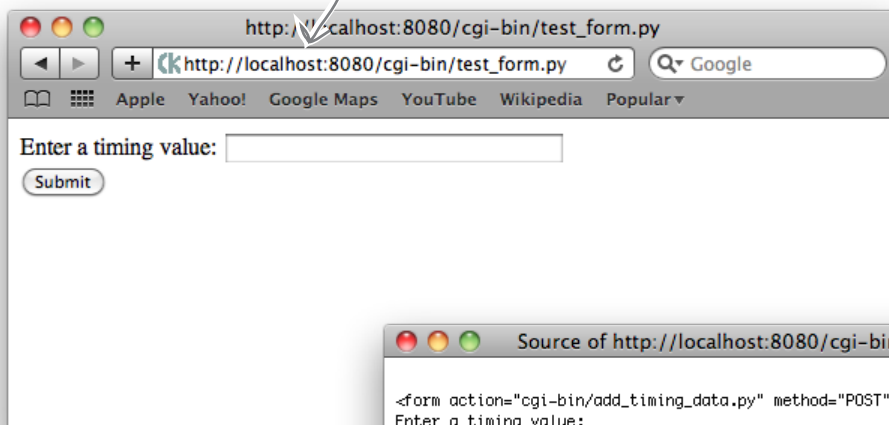
print(yate.start_response('text/html'))
print(yate.do_form('add_timing_data.py', ['TimeValue'], text='Send'))
```

Always start with a CGI response.

Dynamically create the form, supplying any arguments as required.

Set the executable bit (if required on your OS) using `chmod +x test_form.py`, and then use your browser to confirm that your HTML form-generating code is working.

Enter the URL for the CGI script into your web browser's location bar.



Use your browser's "View Source" menu option to confirm that the generated form is exactly what you need.

The generated HTML form appears within the browser's window.



Great. You've extended `yate.py` to support the creation of a simple data entry form. Now all you need to do is to decide what happens once the data arrives on your server.

The data is delivered to your CGI script

In addition to running your webapp, the web server also arranges to deliver any submitted form data to your waiting CGI script. Python's `cgi` library converts the data into a dictionary and, as you already know, provides you with convenient access to the submitted data:

```
import cgi

form = cgi.FieldStorage()
```

All of your form's data has been added to the "form" dictionary.

Additional information about the web request is also available to you via the web server's *environment*. Typically, you won't need to access or use this data directly. However, occasionally, it can be useful to report on some of it.

Here is some code that takes advantage of Python's built-in support for querying your CGI script's environment using the `os` library, assuming the environment values have been set by a friendly web server. Note that the data in the environment is available to your code as a dictionary.

```
import os
import time
import sys
```

Be sure to include the "os" library in your list of imports.

```
addr = os.environ['REMOTE_ADDR']
host = os.environ['REMOTE_HOST']
method = os.environ['REQUEST_METHOD']
```

Query three environment variables and assign their values to variables.

```
cur_time = time.asctime(time.localtime())
```

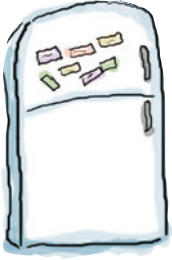
Get the current time.

```
print(host + ", " + addr + ", " + cur_time + ": " + method, file=sys.stderr)
```

Display the queried data on standard error.

Let's exploit both code snippets on this page to **log** the data sent from a form to your web server's console. When you are convinced that the data is arriving at your web server *intact*, you can extend your code to store the received data in your model.

Let's write a CGI to display your form's data.



CGI Magnets

You need a new CGI script called `add_timing_data.py`, which processes the data from a form and displays the data on your web server's console screen. The CGI needs to query the environment, arranging to display the logged data on one line. The code exists, but most of it is all over the floor. Rearrange the magnets to produce a working program.

Don't forget this line if you are running on Max OS X or Linux.

```
#!/usr/local/bin/python3
```

```
import cgi
import os
import time
import sys
import yate
```

There's not much of a response for now...so just send back plain text to the waiting web browser.

There's really nothing new here.

```
print(yate.start_response('text/plain'))
addr = os.environ['REMOTE_ADDR']
host = os.environ['REMOTE_HOST']
method = os.environ['REQUEST_METHOD']
cur_time = time.asctime(time.localtime())
print(host + ", " + addr + ", " + cur_time + ": " + method + ": ",
      end='', file=sys.stderr)
```

```
print('OK.')
```

```
end=' ',
```

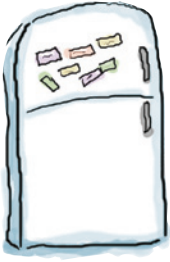
```
form = cgi.FieldStorage()
```

```
print(each_form_item + '->' + form[each_form_item].value,
```

```
print(file=sys.stderr)
```

```
for each_form_item in form.keys():
```

```
file=sys.stderr)
```



CGI Magnets Solution

You need a new CGI script called `add_timing_data.py`, which processes the data from a form and displays the data on your web server's console screen. The CGI needs to query the environment, arranging to display the logged data on one line. The code exists, but most of it is all over the floor. You were to rearrange the magnets to produce a working program.

```
#!/usr/local/bin/python3

import cgi
import os
import time
import sys
import yate

print(yate.start_response('text/plain'))
addr = os.environ['REMOTE_ADDR']
host = os.environ['REMOTE_HOST']
method = os.environ['REQUEST_METHOD']
cur_time = time.asctime(time.localtime())
print(host + ", " + addr + ", " + cur_time + ": " + method + ": ",
      end='', file=sys.stderr)

form = cgi.FieldStorage()
for each_form_item in form.keys():
    print(each_form_item + '->' + form[each_form_item].value,
          end=' ', file=sys.stderr)
print(file=sys.stderr)
print('OK.')
```

Ensure that this "print()" function does NOT take a newline.

Take a newline on standard error.



Test Drive

Let's use your form-generating CGI script from earlier to try out `add_timing_data.py`. As you enter data in the form and press the **Send** button, watch what happens on the web server's console.

Enter some data into your web form. →

http://localhost:8080/cgi-bin/test_form.py

Enter a timing value:

http://localhost:8080/cgi-bin/add_timing_data.py

OK.

The web browser displays a very basic response. All is "OK".

The web server's logging screen displays the data that arrived, as well as the name associated with it.

```
File Edit Window Help DisplayingPOSTs
$ python3 simple_httpd.py
Starting simple_httpd on port: 8080
localhost - - [21/Sep/2010 17:34:54] "GET /cgi-bin/test_form.py HTTP/1.1" 200 -
localhost - - [21/Sep/2010 17:34:54] "GET /favicon.ico HTTP/1.1" 200 -
localhost - - [21/Sep/2010 17:35:24] "POST /cgi-bin/add_timing_data.py HTTP/1.1" 200 -
localhost, 127.0.0.1, Tue Sep 21 17:35:24 2010: POST: TimeValue->2.22
```

That worked perfectly. The data entered into the form is delivered to your CGI script on your server. Your next challenge is to provide the same user input experience on an Android phone.

Ask for input on your Android phone

When you ask for user input on Android, the dialog that your user sees looks something like this example, which asks your user to confirm or change the web address and port for your server.



A single Android call creates this interface for you using the `dialogGetInput()` method:

```
title = 'Which server should I use?'
message = "Please confirm the server address/name to use for your athlete's timing data:"
data = 'http://192.168.1.33:8080'
resp = app.dialogGetInput(title, message, data).result
```

Pressing the **Ok** button sets `resp` to the data entered into the input area.

Pressing the **Cancel** button sets `resp` to `None`, which is Python's internal null-value.

The result of your user's interaction with the dialog is assigned to "resp".

Let's create some Android data-entry dialogs.



Exercise

Let's create a small Android app that interacts with your user *twice*. The first dialog asks the user to confirm the web address and port to use for the web server. Assuming your user taps the OK button on your dialog, a second dialog pops up to request the timing value to send to the server. As with the first dialog, tapping the OK button continues execution by sending the newly acquired timing value to the web server. Tapping Cancel at any time causes your app to exit.

Some of the code is provided for you. Your job is to complete the program. Write the code you think you need under this code, and call your program `get2inputsapp.py`:

There's nothing new here...you've seen all of this code before.

```
import android
from urllib import urlencode
from urllib2 import urlopen

server_title = 'Which server should I use?'
server_msg = "Please confirm the server address/name to use for your athlete's timing data:"
timing_title = 'Enter data'
timing_msg = 'Provide a new timing value:'
web_server = 'http://192.168.1.33:8080'
add_time CGI = '/cgi-bin/add_timing_data.py'

app = android.Android()

def send_to_server(url, post_data=None):
    if post_data:
        page = urlopen(url, urlencode(post_data))
    else:
        page = urlopen(url)
    return (page.read().decode("utf8"))
```

```
.....
.....
.....
.....
.....
.....
.....
.....
```



Exercise Solution

You were to create a small Android app that interacts with your user *twice*. The first dialog asks the user to confirm the web address and port to use for the web server. Assuming your user taps the OK button on your dialog, a second dialog pops up to request the timing value to send to the server. As with the first dialog, tapping the OK button continues execution by sending the newly acquired timing value to the web server. Tapping Cancel at any time causes your app to exit.

Some of the code was provided for you. Your job was to complete the program by writing the code you think you need under this code and call your program `get2inputsapp.py`.

```
import android
from urllib import urlencode
from urllib2 import urlopen

server_title = 'Which server should I use?'
server_msg = "Please confirm the server address/name to use for your athlete's timing data:"
timing_title = 'Enter data'
timing_msg = 'Provide a new timing value:'
web_server = 'http://192.168.1.33:8080'
add_time CGI = '/cgi-bin/add_timing_data.py'

app = android.Android()

def send_to_server(url, post_data=None):
    if post_data:
        page = urlopen(url, urlencode(post_data))
    else:
        page = urlopen(url)
    return (page.read().decode("utf8"))

.....
resp = app.dialogGetInput(server_title, server_msg, web_server).result
.....

.....
if resp is not None:
    web_server = resp
    resp = app.dialogGetInput(timing_title, timing_msg).result
    .....
    if resp is not None:
        new_time = resp
        send_to_server(web_server + add_time CGI, {'TimingValue': new_time})
    .....
.....
```

The first dialog asks your user to confirm the web address and port to use.

If your user did NOT tap on the Cancel button...

...the second dialog asks for a new timing value.

Again, if your user did NOT tap on the Cancel button...

...the app sends the data to the web server.

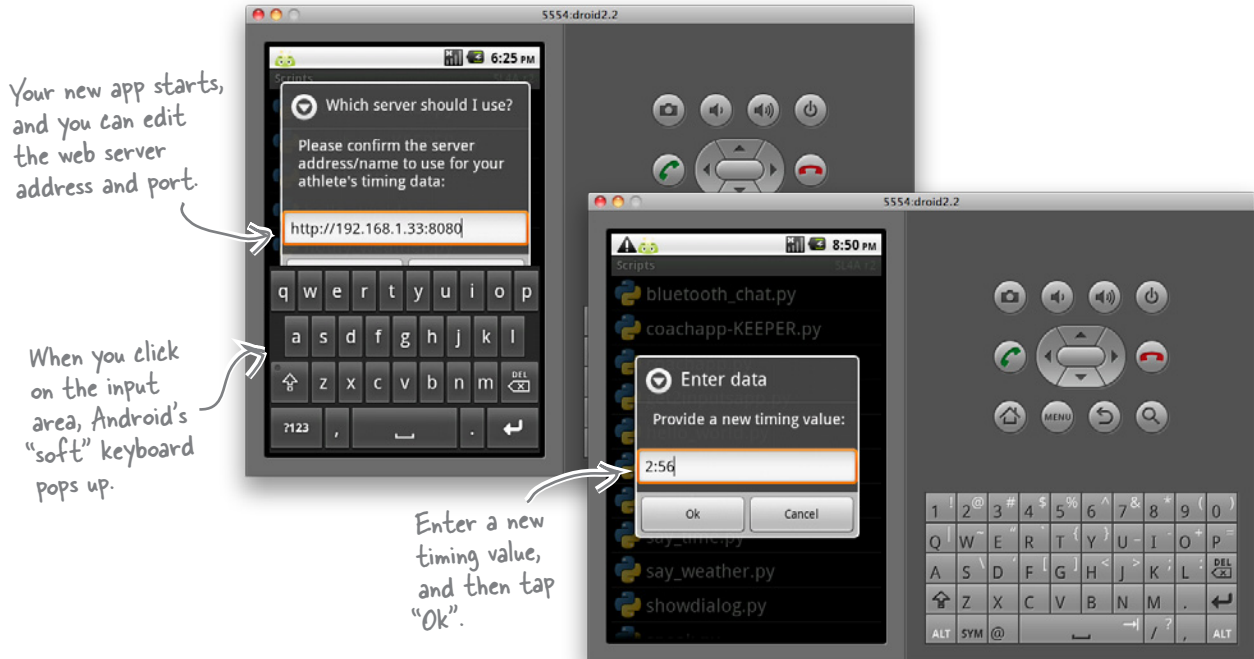


Test Drive

Let's copy `get2inputsapp.py` to the emulator using the `adb` tool:

```
tools/adb push get2inputsapp.py /sdcard/sl4a/scripts
```

The `get2inputsapp.py` app appears on the list of scripts within SL4A. Go ahead and give it a tap:



```
File Edit Window Help InputsFromAndroid
$ python3 simple_httpd.py
Starting simple_httpd on port: 8080
localhost - - [21/Sep/2010 17:34:54] "GET /cgi-bin/test_form.py HTTP/1.1" 200 -
localhost - - [21/Sep/2010 17:34:54] "GET /favicon.ico HTTP/1.1" 200 -
localhost - - [21/Sep/2010 17:35:24] "POST /cgi-bin/add_timing_data.py HTTP/1.1" 200 -
localhost, 127.0.0.1, Tue Sep 21 17:35:24 2010: POST: TimeValue->2.22
192.168.1.33 - - [21/Sep/2010 20:50:30] "POST /cgi-bin/add_timing_data.py HTTP/1.1" 200 -
localhost, 192.168.1.33, Tue Sep 21 20:50:30 2010: POST: TimingValue->2:56
```

The web server's log confirms the data was sent from your emulator.

Perfect. That's working, too. Regardless of where your data originates—on the Web or a phone—your app can send it to your web server.

It's time to update your server data



Which of your two datasets should you update?

If you update the pickle, the next time the `put_to_store()` function runs, your most recent update will vanish as `put_to_store()` recreates the pickle from the data in the text files. That's not good.

If you update the appropriate athlete's text file, the data in the pickle will be **stale** until `put_to_store()` runs again. If another process calls the `get_from_store()` function in the meantime, the update to the pickle might not have been applied and will appear to be *missing* for anyone reading your data. That's not good, either.



Avoid race conditions



Of course...I could write to the text file and then immediately call "put_to_store()" to update the pickle, right?

Yes, that's one possible solution, but it's a poor one.

You might think it highly unlikely...but it is possible for another process to call the `get_from_store()` function *between* the text file update and the pickle recreation, resulting in a short period of data inconsistency. These types of situations are known as **race conditions** and are hard to debug when they occur.

It's best to keep them from ever happening if you can.

The basic problem here is that you have *one* update with *one* piece of data that results in *two* file interactions. If nothing else, that's just wasteful.



Your up-to-date text file

Hey, thanks for the update!

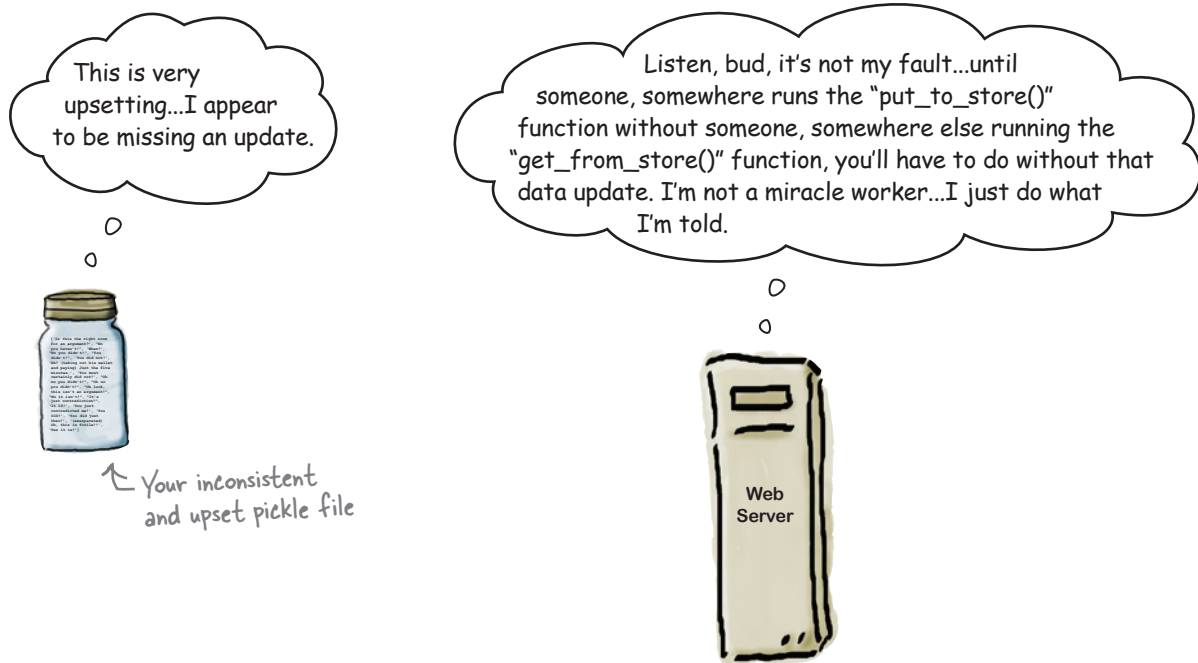


↑ Your temporarily inconsistent pickle file

What update?!?
It's been all quiet over here...

You need a better data storage mechanism

Your initial text files and pickle design is fine when only one user is accessing the data. However, now that more than one person can access the data at any time, and from anywhere, your design is in need of improvement. Above all, you need to *avoid that race condition*.



there are no Dumb Questions

Q: Surely you should have thought about this problem long ago and designed this “properly” from the start?

A: That’s certainly one way to look at things, and hindsight is always a wonderful thing! However, programs have a tendency to start out small, then grow to provide more features, which can introduce complexity. Recall that the coach’s app started life as a simple “standalone” text-based program, which was then moved to the Web to support multiple users. Part of the app was then redeveloped for use on an Android phone. And yes, if we’d known all of this ahead of time, we might have been designed it differently.

Q: So I’m facing a rewrite of large chunks of my code?

A: Let’s see. You did build your program using the MVC pattern, and you are using Python, so those two facts should take the sting out of any potential rewrite, assuming a rewrite is what’s required here.



Wouldn't it be dreamy if I could put my data in only one place and support all my app's requirements? But I know it's just a fantasy...

Use a database management system

You need to move away from your text file and pickle combination and use a *real* database management system. You have plenty of choices here...



All of these fine technologies will work, but they are overkill for your app's data requirements. And besides some of these are way beyond the NUAC's budget, let alone their ability to set up, run, and maintain such a system.

What you need is something that's effectively hidden from the NUAC yet lets you take advantage of what a database management system has to offer.

If only such a technology existed...

Python includes SQLite

Python 3 comes preinstalled with Release 3 of **SQLite**, a full-featured, zero-config, SQL-based data management system.

To use SQLite, simply import the `sqlite3` library and use Python's standardized database API to program it. There's really nothing to it: no database setup, no config, and no ongoing maintenance.

With your data stored in SQLite, rewrite your webapp's model code to use SQL to access, manipulate, and query your data. You can plan to move to one of the bigger database systems if and when your application needs dictate such a move.

SQLite sounds perfect for the NUAC's data, doesn't it?



Geek Bits

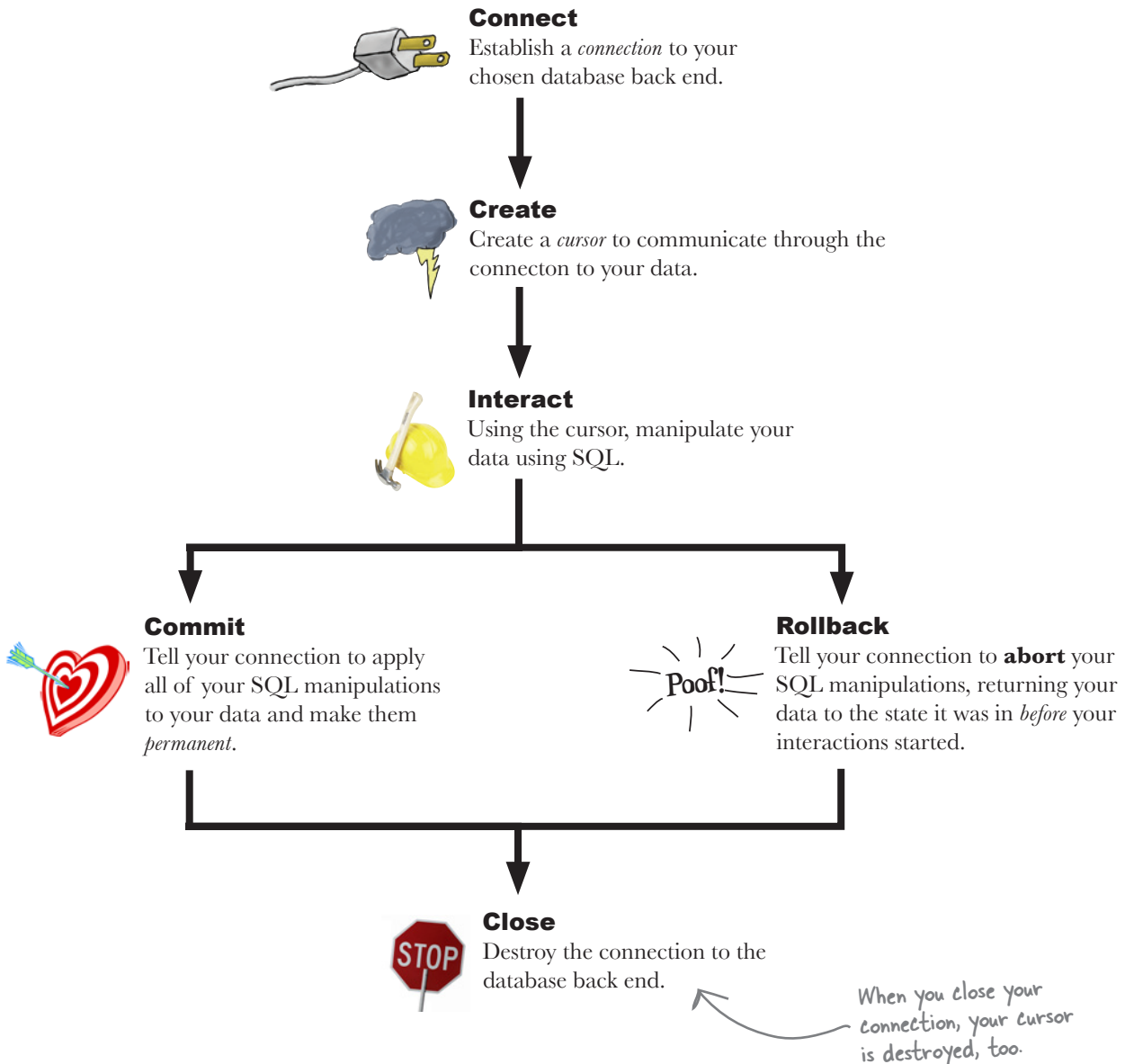


The material in this chapter assumes you are comfortable with SQL database technology. If you are new to SQL (or just need a quick refresher), take a look at *Head First SQL*, which comes **highly recommended**.

[Note from Marketing: Available anywhere good books are sold and to anyone with a valid credit card.]

Exploit Python's database API

The **Python Database API** provides a standard mechanism for programming a wide variety of database management systems, including SQLite. The process you follow in your code is the same regardless of which back-end database you're using.



The database API as Python code

Here's how to implement an interaction with a database using the `sqlite3` module:

The diagram illustrates a sequence of Python code snippets for interacting with a SQLite database. Each line of code is accompanied by a handwritten annotation on the left, with an arrow pointing to the corresponding code. On the right, a series of icons represent the steps: a plug for connection, a lightning bolt for cursor creation, a hammer for SQL execution, a heart for commit, and a stop sign for closing the connection. A note at the top right explains the database file.

As always, import the library you need. → `import sqlite3`

Establish a connection to a database. → `connection = sqlite3.connect('test.sqlite')`

Create a cursor to the data. → `cursor = connection.cursor()`

Execute some SQL. → `cursor.execute("""SELECT DATE('NOW')""")`

Commit any changes, making them permanent. → `connection.commit()`

Close your connection when you're finished. → `connection.close()`

This disk file is used to hold the database and its tables.

Icons: plug, lightning bolt, hammer, heart, stop sign.

Depending on what happens during the **Interact** phase of the process, you either make any changes to your data permanent (`commit`) or decide to abort your changes (`rollback`).

You can include code like this in your program. It is also possible to interact with you SQLite data from within IDLE's shell. Whichever option you choose, you are interacting with your database using Python.

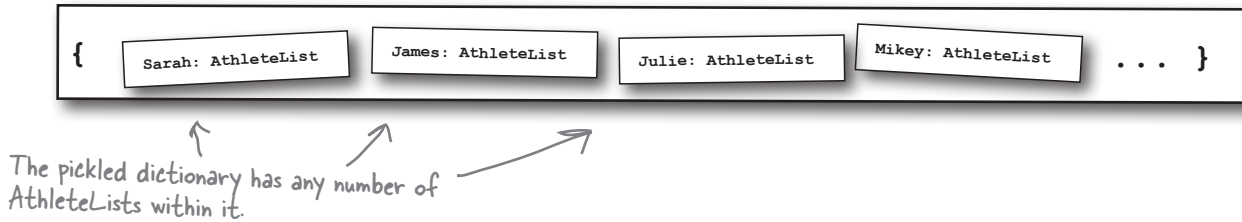
It's great that you can use a database to hold your data. But what schema should you use? Should you use one table, or do you need more? What data items go where? How will you design your database?

Let's start working on the answers to these questions.

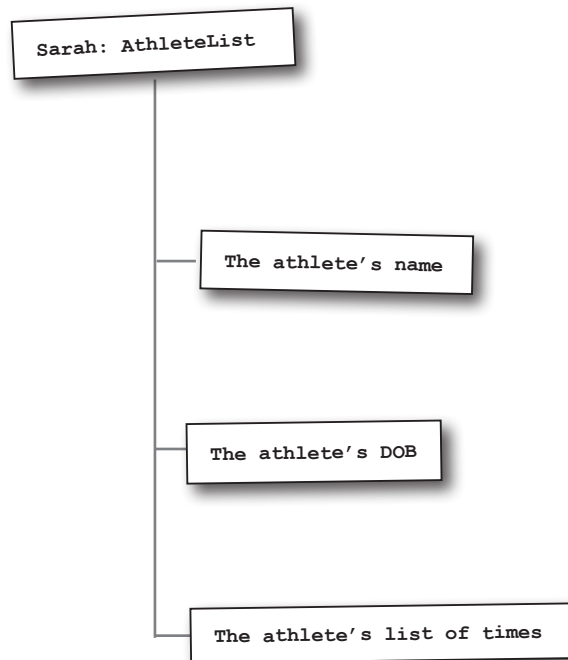
A little database design goes a long way

Let's consider how the NUAC's data is currently stored within your pickle.

Each athlete's data is an `AthleteList` object instance, which is associated with the athlete's name in a dictionary. The entire dictionary is pickled.



Each `AthleteList` has the following attributes:



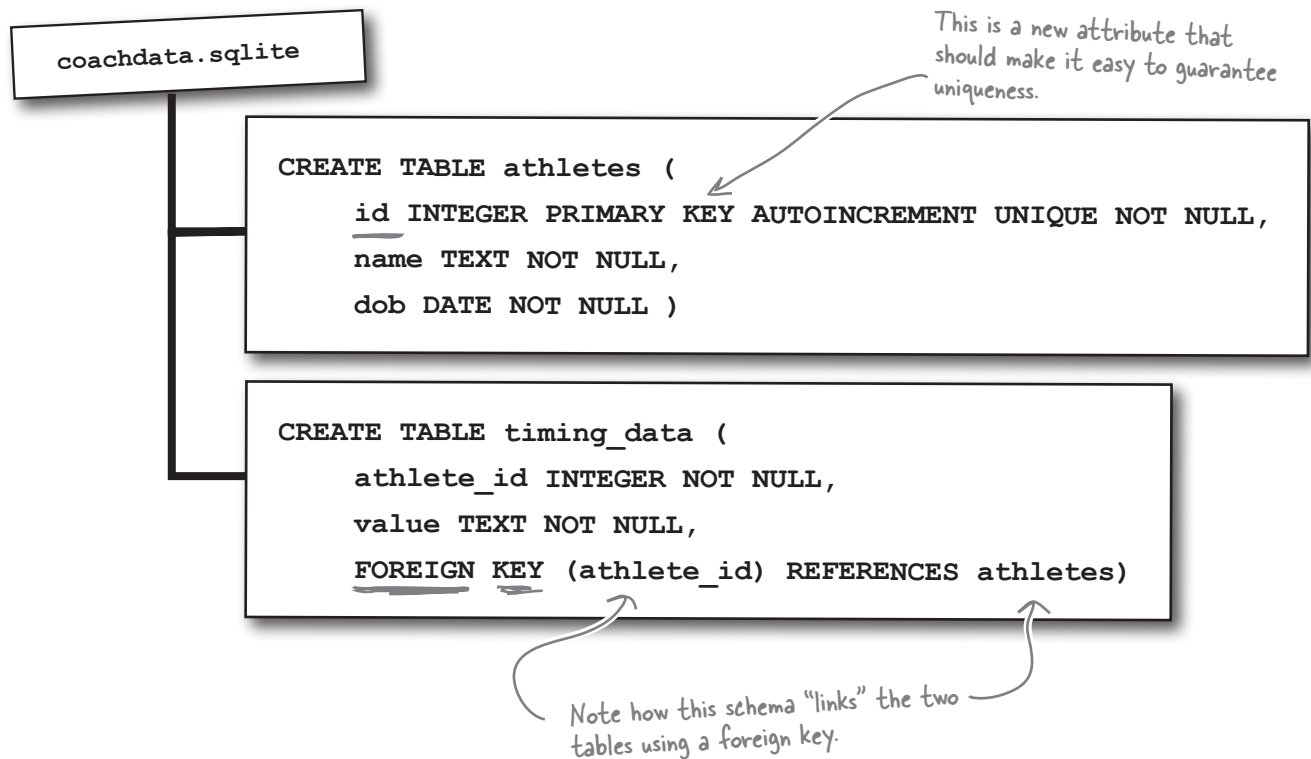
With this arrangement, it is pretty obvious which name, date of birth, and list of times is associated with which individual athlete. But how do you model these relationships within a SQL-compliant database system like *SQLite*?

You need to define your schema and create some tables.

Define your database schema

Here is a suggested SQL schema for the NUAC's data. The database is called `coachdata.sqlite`, and it has two related tables.

The first table, called `athletes`, contains rows of data with a unique ID value, the athlete's name, and a date-of-birth. The second table, called `timing_data`, contains rows of data with an athlete's unique ID and the actual time value.



There can be one *and only one* row of data for each athlete in the `athletes` table. For each athlete, the value of `id` is guaranteed to be *unique*, which ensures that two (or more) athletes with the same name are kept separate within the system, because that have different ID values.

Within the `timing_data` table, each athlete can have any number of time values associated with their unique `athlete_id`, with an individual row of data for each recorded time.

Let's look at some sample data.

What does the data look like?

If the two tables were created and then populated with the data from the NUAC's text files, the data in the tables might look something like this.

id	name	dob
1	James Lee	2002-03-14
2	Sarah Sweeney	2002-06-17
3	Vera Vi	2002-12-25
4	Julie Jones	2002-08-17
5	Sally Sanchez	2002-11-24
6	Mikey McManus	2002-02-24

This is what the data in the "athletes" table might look like, with one row of data for each athlete.

This is what the data in the "timing_data" table might look like, with multiple rows of data for each athlete and one row for each timing value.

athlete_id	value
1	2.01
1	2.16
1	2.22
1	2.34
1	2.45
1	3.01
1	3.1
1	3.21
2	2.18
2	2.21
2	2.22
2	2.25
2	2.39
2	2.54
2	2.55
2	2.58
3	2.41
3	2.49
3	3.01
3	3.02
3	3.11
3	3.23
4	2.11
4	2.23

There's more data in this table than shown here.

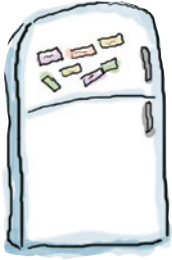
If you create these two tables then arrange for your data to be *inserted* into them, the NUAC's data would be in a format that should make it easier to work with.

Looking at the tables, it is easy to see how to add a new timing value for an athlete. Simply add another row of data to the `timing_data` table.

Need to add an athlete? Add a row of data to the `athletes` table.

Want to know the fastest time? Extract the smallest value from the `timing_data` table's `value` column?

Let's create and populate these database tables.



SQLite Magnets

Let's create a small Python program that creates the `coachdata.sqlite` database with the empty `athletes` and `timing_data` tables. Call your program `createDBtables.py`. The code you need is almost ready. Rearrange the magnets at the bottom of the page to complete it.

```
import sqlite3
```

```
.....

.....

cursor.execute("""CREATE TABLE athletes (

.....

.....

.....

.....

athlete_id INTEGER NOT NULL,
value TEXT NOT NULL,
FOREIGN KEY (athlete_id) REFERENCES athletes)""")

connection.commit()
connection.close()
```

```
dob DATE NOT NULL )""")
```

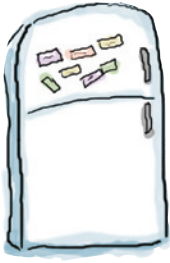
```
cursor = connection.cursor()
```

```
cursor.execute("""CREATE TABLE timing_data (
```

```
connection = sqlite3.connect('coachdata.sqlite')
```

```
name TEXT NOT NULL,
```

```
id INTEGER PRIMARY KEY AUTOINCREMENT UNIQUE NOT NULL,
```



SQLite Magnets Solution

Your job was to create a small Python program that creates the `coachdata.sqlite` database with the empty `athletes` and `timing_data` tables. You were to call your program `createdBtables.py`. The code you needed was almost ready, and you were to rearrange the magnets at the bottom of the page to complete it.

```
import sqlite3
```



```
connection = sqlite3.connect('coachdata.sqlite')
```



```
cursor = connection.cursor()
```

```
cursor.execute("""CREATE TABLE athletes (
```

```
    id INTEGER PRIMARY KEY AUTOINCREMENT UNIQUE NOT NULL,
```

```
    name TEXT NOT NULL,
```

```
    dob DATE NOT NULL )""")
```



```
cursor.execute("""CREATE TABLE timing_data (
```

```
    athlete_id INTEGER NOT NULL,
```

```
    value TEXT NOT NULL,
```

```
    FOREIGN KEY (athlete_id) REFERENCES athletes)""")
```



```
connection.commit()
```



```
connection.close()
```

The `commit` isn't always required with most other database systems, but it is with SQLite.

Transfer the data from your pickle to SQLite

As well as writing the code to create the tables that you need, you also need to arrange to transfer the data from your existing model (your text files and pickle combination) to your new database model. Let's write some code to do that, too.

You can add data to an existing table with the SQL `INSERT` statement. Assuming you have data in variables called `name` and `dob`, use code like this to add a new row of data to the `athletes` table:

```
cursor.execute("INSERT INTO athletes (name, dob) VALUES (?, ?)", (name, dob))
```

The data in these variables is substituted in place of the "?" placeholders.

You don't need to worry about supplying a value for the "id" column, because SQLite provides one for you automatically.



Ready Bake Python Code

Here's a program, called `initDBathletes.py`, which takes your athlete data from your existing model and loads it into your newly created *SQLite* database.

Connect to the new database.

Grab the data from the existing model.

Get the athlete's name and DOB from the pickled data.

Use the `INSERT` statement to add a new row to the "athletes" table.

```
import sqlite3

connection = sqlite3.connect('coachdata.sqlite')
cursor = connection.cursor()

import glob
import athletemodel
data_files = glob.glob("../data/*.txt")
athletes = athletemodel.put_to_store(data_files)

for each_ath in athletes:
    name = athletes[each_ath].name
    dob = athletes[each_ath].dob

    cursor.execute("INSERT INTO athletes (name, dob) VALUES (?, ?)", (name, dob))
    connection.commit()

connection.close()
```

Make the change(s) permanent.

What ID is assigned to which athlete?

You need to query the data in your database table to work out which ID value is automatically assigned to an athlete.

With SQL, the `SELECT` statement is the *query king*. Here's a small snippet of code to show you how to use it with Python, assuming the `name` and `dob` variables have values:

Again, the placeholders indicate where the data values are substituted into the query.

```
cursor.execute("SELECT id from athletes WHERE name=? AND dob=?", (name, dob))
```

If the query succeeds and returns data, it gets added to your cursor. You can call a number of methods on your cursor to access the results:

- `cursor.fetchone()` returns the next row of data.
- `cursor.fetchmany()` returns multiple rows of data.
- `cursor.fetchall()` returns all of the data.

Each of these cursor methods return a list of rows.



Names alone are not enough anymore...if you want to uniquely identify your athletes, I need to know their IDs.

Insert your timing data

You're on a roll, so let's *keep coding* for now and produce the code to take an athlete's timing values out of the pickle and add them to your database. Specifically, you'll want to arrange to add a new row of data to the `timing_data` table for each time value that is associated with each athlete in your pickle.

Those friendly coders over at the Head First Code Review Team have just announced they've added a `clean_data` attribute to your `AthleteList` class. When you access `clean_data`, you get back a list of timing values that are sanitized, sorted, and free from duplicates. The Head First Code Review Team has excellent timing; that attribute should come in handy with your current coding efforts.



Sharpen your pencil

Again, it's OK to assume in your code that the "name" and "dob" variables exist and have values assigned to them.

Grab your pencil and write the lines of code needed to query the `athletes` table for an athlete's name and DOB, assigning the result to a variable called the `_current_id`. Write another query to extract the athlete's `times` from the `pickle` and add them to the `timing` data table.



Sharpen your pencil Solution

You were to grab your pencil and write the lines of code needed to query the `athletes` table for an athlete's name and DOB, assigning the result to a variable called `the_current_id`. You were then to write another query to extract the athlete's times from the pickle and add them to the `timing_data` table.

Query the "athletes" table for the ID.

```
cursor.execute("SELECT id from athletes WHERE name=? AND dob=?",
               (name, dob))
```

Remember: "fetchone()" returns a list.

```
the_current_id = cursor.fetchone()[0]
```

Take each of the "clean" times and use it, together with the ID, within the SQL "INSERT" statement.

```
for each_time in athletes[each_ath].clean_data:
```

```
    cursor.execute("INSERT INTO timing_data (athlete_id, value) VALUES (?, ?)",
                  (the_current_id, each_time))
```

It often makes sense to split your execute statement over multiple lines.

```
connection.commit()
```

As always, make the change(s) permanent.

Add the ID and the time value to the "timing_data" table.



Do this!

Add the code to your `initDBathletes.py` code from earlier, just after the `connection.commit()` call. Rename your program `initDBtables.py`, now that both the `athletes` and `timing_data` tables are populated with data by a single program.

That's enough coding (for now). Let's transfer your pickled data.



Test Drive

You've got two programs to run now: `createDBtables.py` creates an empty database, defining the two tables, and `initDBtables.py` extracts the data from your pickle and populates the tables. Rather than running these programs within IDLE, let's use the Python command-line tool instead.

If you are running Windows,
replace "python3" with this:
"C:\Python31\python.exe".

Be careful
to run both
programs *ONLY*
once.

```
File Edit Window Help PopulateTheTables
$ python3 createDBtables.py
$ python3 initDBtables.py
$
```



Hello? Something happened there,
didn't it? I ran the programs but nothing
appeared on screen...how do I know if
anything worked?

SQLite data management tools

When it comes to checking if your manipulations of the data in your database worked, you have a number of options:

- a** Write more code to check that the database is in the state that you expect it.

Which can certainly work, but is error-prone, tedious, and way too much work.

Life really is too short.

- b** Use the supplied “sqlite3” command-line tool.

Simply type `sqlite3` within a terminal window to enter the SQLite “shell.” To find out which commands are available to you, type `.help` and start reading. The tool is a little basic (and cryptic), but it works.

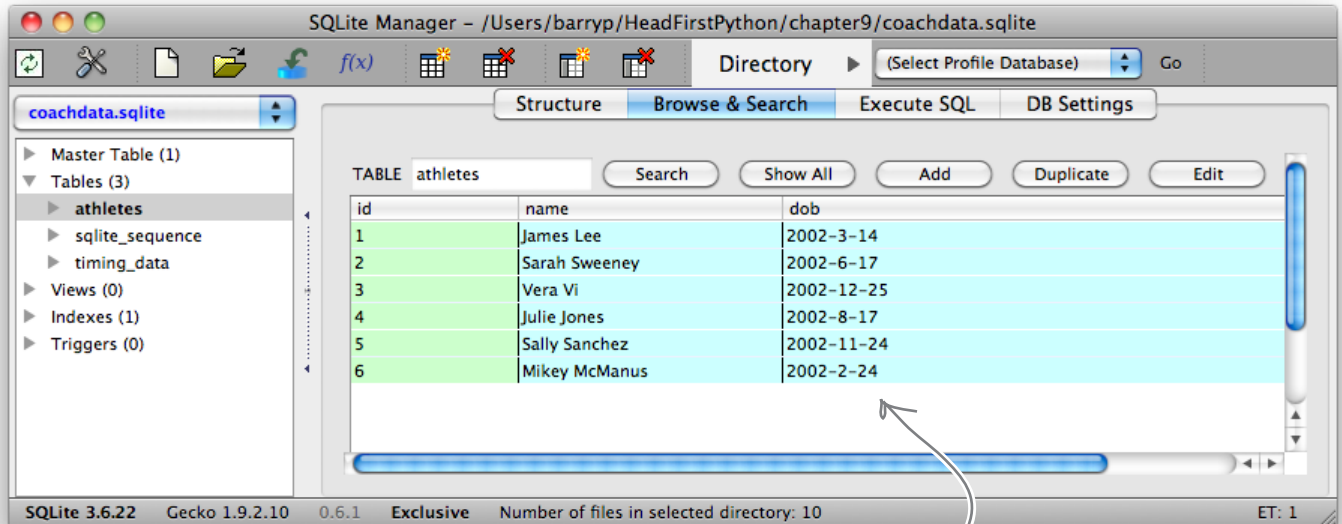
That's a period, followed by the word “help”.

- c** Use a graphical database browser.

There are lots of these; just Google “sqlite database browser” for more choices than you have time to review. Our favorite is the SQLite Manager, which installs into the Firefox web browser as an extension.

Works great, but only on Firefox.

This is what SQLite Manager looks like.



Great, all of the athletes are in the “athletes” table.

But how do you integrate your new database into your webapp?

Integrate SQLite with your existing webapp



Joe: This should be easy. We just have to rewrite the code in `athletemodel.py` to use the database, while keeping the API the same.

Frank: What do you mean by *keeping the API the same*?

Joe: Well...take the `get_from_store()` function, for instance. It returns an `AthleteList` dictionary, so we need to make sure that when we update `get_from_store()` to use our database that it *continues* to return a dictionary, just as it's always done.

Frank: Ah, now I get it: we can query the database, grab all the data, turn it into a big dictionary containing all of our `AthleteList` objects and then return that to the caller, right?

Joe: Yes, exactly! And the best of it is that the calling code doesn't need to change at all. Don't you just love the beauty of MVC?

Frank: Ummm...I guess so.

Jim: [cough, cough]

Frank: What's up, Jim?

Jim: Are you guys crazy?

Joe & Frank: What?!?

Jim: You are bending over backward to maintain compatibility with an API that exists only because of the way your data model was initially designed. Now that you've reimplemented how your data is stored in your model, you need to consider if you need to change your API, too.

Joe & Frank: Change our API? *Are you crazy?!?*

Jim: No, not crazy, just pragmatic. If we can simplify the API by redesigning it to better fit with our database, then we should.

Joe: OK, but we haven't got all day, y'know.

Jim: Don't worry: it'll be worth the effort.



Long Exercise

Let's spend some time amending your model code to use your SQLite database as opposed to your pickle. Start with the code to your `athletemodel.py` module. Take a pencil and strike out the lines of code you no longer need.

```
import pickle

from athletemodel import AthleteList

def get_coach_data(filename):
    try:
        with open(filename) as f:
            data = f.readline()
            templ = data.strip().split(',')
            return(AthleteList(templ.pop(0), templ.pop(0), templ))
    except IOError as ioerr:
        print('File error (get_coach_data): ' + str(ioerr))
        return(None)

def put_to_store(files_list):
    all_athletes = {}
    for each_file in files_list:
        ath = get_coach_data(each_file)
        all_athletes[ath.name] = ath
    try:
        with open('athletes.pickle', 'wb') as athf:
            pickle.dump(all_athletes, athf)
    except IOError as ioerr:
        print('File error (put_and_store): ' + str(ioerr))
    return(all_athletes)
```

```
def get_from_store():
    all_athletes = {}
    try:
        with open('athletes.pickle', 'rb') as athf:
            all_athletes = pickle.load(athf)
    except IOError as ioerr:
        print('File error (get_from_store): ' + str(ioerr))
    return(all_athletes)

def get_names_from_store():
    athletes = get_from_store()
    response = [athletes[each_ath].name for each_ath in athletes]
    return(response)
```

Remember: there's no requirement to maintain the existing API.





Long Exercise Solution

Let's spend some time amending your model code to use your SQLite database as opposed to your pickle. Start with the code to your `athletemodel.py` module. You were to take a pencil and strike out the lines of code you no longer need.

```
import pickle

from athleteList import AthleteList

def get_coach_data(filename):
    try:
        with open(filename) as f:
            data = f.readline()
            templ = data.strip().split(',')
            return(AthleteList(templ.pop(0), templ.pop(0), templ))
except IOError as ioerr:
    print('File error (get_coach_data): ' + str(ioerr))
    return(None)

def put_to_store(files_list):
    all_athletes = {}
    for each_file in files_list:
        ath = get_coach_data(each_file)
        all_athletes[ath.name] = ath
    try:
        with open('athletes.pickle', 'wb') as athf:
            pickle.dump(all_athletes, athf)
except IOError as ioerr:
    print('File error (put_and_store): ' + str(ioerr))
    return(all_athletes)
```

None of this code is needed anymore, because SQLite provides the data model for you.

```
def get_from_store():  
    all_athletes = {}  
    try:  
        with open('athletes.pickle', 'rb') as athf:  
            all_athletes = pickle.load(athf)  
    except IOError as ioerr:  
        print('File error (get_from_store): ' + str(ioerr))  
    return(all_athletes)  
  
def get_names_from_store():  
    athletes = get_from_store()  
    response = [athletes[each_ath].name for each_ath in athletes]  
    return(response)
```



This might seem a little drastic...but sometimes a redesign requires you to throw away obsolete code.

You still need the list of names

Throwing away all of your “old” model code makes sense, but you still need to generate a list of names from the model. Your decision to use SQLite is about to pay off: all you need is a simple SQL SELECT statement.



Ready Bake Python Code

Here's the code for your new `get_names_from_store()` function:

Connect to the
database.

Extract the
data you need.

Formulate a
response.

```
import sqlite3

db_name = 'coachdata.sqlite'

def get_names_from_store():
    connection = sqlite3.connect(db_name)
    cursor = connection.cursor()
    results = cursor.execute("""SELECT name FROM athletes""")
    response = [row[0] for row in results.fetchall()]
    connection.close()
    return(response)
```

Return the list of
names to the caller.

I guess in this case it
actually makes perfect
sense to maintain the
API for this call.

o o



Get an athlete's details based on ID

In addition to the list of names, you need to be able to extract an athlete's details from the `athletes` table based on ID.



Ready Bake Python Code

Here's the code for another new function called `get_athlete_from_id()`:

A new function gets the data associated with a specific ID.

Get the "name" and "DOB" values from the `athletes` table.

Get the list of times from the "timing_data" table.

Return the athlete's data to the caller.

```
def get_athlete_from_id(athlete_id):
    connection = sqlite3.connect(db_name)
    cursor = connection.cursor()

    results = cursor.execute("""SELECT name, dob FROM athletes WHERE id=?""",
                              (athlete_id,))
    (name, dob) = results.fetchone()

    results = cursor.execute("""SELECT value FROM timing_data WHERE athlete_id=?""",
                              (athlete_id,))
    data = [row[0] for row in results.fetchall()]

    response = {
        'Name': name,
        'DOB': dob,
        'data': data,
        'top3': data[0:3]}

    connection.close()
    return(response)
```

Note the use of the placeholder to indicate where the "athlete_id" argument is inserted into the SQL SELECT query.

Take the data from both query results and turn it into a dictionary.

This function is a more involved than `get_names_from_store()`, but not by much. It still follows the API used with working with data stored in SQLite. This is coming along, nicely.

With the model code converted, you can revisit your CGI scripts to use your new model API.

Let's see what's involved with converting the CGIs.



Isn't there a problem here? The "get_names_from_store()" function returns a list of names, while the "get_athlete_from_id()" function expects to be provided with an ID. But how does the web browser or the phone know which ID to use when all it has to work with are the athletes' names?

That's a good point: which ID do you use?

Your current CGIs all operate on the athlete name, not the ID. In order to ensure each athlete is **unique**, you designed your database schema to include a unique ID that allows for your system to properly identify two (or more) athletes with the *same name*, but at the moment, your model code doesn't provide the ID value to either your web browser or your phone.

One solution to this problem is to ensure that the athlete names are displayed to the user within the view, while the IDs are used *internally* by your system to unique identify a specific athlete. For this to work, you need to change `get_names_from_store()`.

**Exercise**

Here is the current code for your `get_names_from_store()` function. Rather than amending this code, create a new function, called `get_namesID_from_store()`, based on this code but including the ID values as well as the athlete names in its response. Write your new function in the space provided.

```
import sqlite3

db_name = 'coachdata.sqlite'

def get_names_from_store():
    connection = sqlite3.connect(db_name)
    cursor = connection.cursor()
    results = cursor.execute("""SELECT name FROM athletes""")
    response = [row[0] for row in results.fetchall()]
    connection.close()
    return(response)
```

.....

.....

.....

.....

.....

.....

.....

.....

.....

.....

.....

.....

.....

.....

.....



Exercise Solution

Here is your current code for your `get_names_from_store()` function. Rather than amending this code, you were to create a new function, called `get_namesID_from_store()`, based on this code but including the ID values as well as the athlete names in its response. You were to write your new function in the space provided.

```
import sqlite3

db_name = 'coachdata.sqlite'

def get_names_from_store():
    connection = sqlite3.connect(db_name)
    cursor = connection.cursor()
    results = cursor.execute("""SELECT name FROM athletes""")
    response = [row[0] for row in results.fetchall()]
    connection.close()
    return(response)
```

```
def get_namesID_from_store():
```

```
    connection = sqlite3.connect(db_name)
```

```
    cursor = connection.cursor()
```

```
    results = cursor.execute("""SELECT name, id FROM athletes""")
```

```
    response = results.fetchall()
```

```
    connection.close()
```

```
    return(response)
```

Arrange to include the value of "id" in the SQL "SELECT" query.

There's no need to process "results" in any way...assign everything returned from the query to "response".

Remember: when you close your connection, your cursor is also destroyed, so you'll generate an exception if you try and use "return(results.fetchall())".



This is the "generate_list.py"
CGI script.

```
#!/usr/local/bin/python3

import glob
import athletemodel
import yate

data_files = glob.glob("data/*.txt")
athletes = athletemodel.put_to_store(data_files)

print(yate.start_response())
print(yate.include_header("NUAC's List of Athletes"))
print(yate.start_form("generate_timing_data.py"))
print(yate.para("Select an athlete from the list to work with:"))
for each_athlete in sorted(athletes):
    print(yate.radio_button("which_athlete", athletes[each_athlete].name))
print(yate.end_form("Select"))
print(yate.include_footer({"Home": "/index.html"}))
```

Note the change to
the title.

This is "generate_timing_data.py".

```
#!/usr/local/bin/python3

import cgi
import athletemodel
import yate

athletes = athletemodel.get_from_store()
form_data = cgi.FieldStorage()
athlete_name = form_data['which_athlete'].value

print(yate.start_response())
print(yate.include_header("NUAC's Timing Data"))
print(yate.header("Athlete: " + athlete_name + ", DOB: " + athletes[athlete_name].dob + "."))
print(yate.para("The top times for this athlete are:"))
print(yate.u_list(athletes[athlete_name].top3))
print(yate.para("The entire set of timing data is: " + str(athletes[athlete_name].clean_data) +
    " (duplicates removed)."))
print(yate.include_footer({"Home": "/index.html", "Select another athlete": "generate_list.py"}))
```

Another title change.

This "Sharpen" is continued
on the next page, but no
peeking! Don't flip over until
you've amended the code on
this page.



Part 2: You're not done with that pencil just yet! In addition to amending the code to the CGIs that support your web browser's UI, you also need to change the CGIs that provide your webapp data to your Android app. Amend these CGIs, too.

This is the
"generate_names.py"
CGI.

```
#!/usr/local/bin/python3
import json

import athletemodel
import yate

names = athletemodel.get_names_from_store()

print(yate.start_response('application/json'))
print(json.dumps(sorted(names)))
```

And here is the
"generate_data.py"
CGI.

```
#!/usr/local/bin/python3
import cgi
import json
import sys

import athletemodel
import yate

athletes = athletemodel.get_from_store()

form_data = cgi.FieldStorage()
athlete_name = form_data['which_athlete'].value

print(yate.start_response('application/json'))
print(json.dumps(athletes[athlete_name].as_dict))
```

Sharpen your pencil Solution

This is the "generate_list.py"
CGI script.

```
#!/usr/local/bin/python3
```

```
import glob  
import athletemodel  
import yate
```

You no longer need the "glob" module,
as "get_name/D_from_store()" does
all this work for you.

```
data_files = glob.glob("data/*.txt")  
athletes = athletemodel.get_names/D_from_store()
```

```
print(yate.start_response())  
print(yate.include_header("NUAC's List of Athletes"))  
print(yate.start_form("generate_timing_data.py"))  
print(yate.para("Select an athlete from the list to work with:"))  
for each_athlete in sorted(athletes):  
    print(yate.radio_button("which_athlete", athletes[each_athlete].name))  
print(yate.end_form("Select"))  
print(yate.include_footer({"Home": "/index.html"}))
```

The "athletes" are now a list of
lists, so amend the code to get
at the data you need.

each_athlete[0], each_athlete[1]

radio_button_id() !?

This is "generate_timing_data.py".

```
#!/usr/local/bin/python3
```

```
import cgi  
import athletemodel  
import yate
```

It looks like you might need
a slightly different "radio_
button()" function!?

```
athletes = athletemodel.get_from_store()  
form_data = cgi.FieldStorage()  
athlete_name = form_data['which_athlete'].value
```

Get the athlete's data
from the model, which
returns a dictionary.

```
athlete = athletemodel.get_athlete_from_id(athlete_id)  
print(yate.start_response())  
print(yate.include_header("NUAC's Timing Data"))  
print(yate.header("Athlete: " + athlete['Name'] + ", DOB: " + athlete['DOB'] + "."))  
print(yate.para("The top times for this athlete are:"))  
print(yate.u_list(athlete['top3']))  
print(yate.para("The entire set of timing data is: " + str(athlete['data']) +  
    " (duplicates removed)."))  
print(yate.include_footer({"Home": "/index.html", "Select another athlete": "generate_list.py"}))
```

Use the returned data as
needed, accessing each of
the dictionary key/values to
get at the athlete's data.

athlete['Name'] + " DOB: " + athlete['DOB']

athlete['top3']

str(athlete['data'])

The rest of this "Sharpen
Solution" is on the next page.



Sharpen your pencil Solution

This is the
"generate_names.py"
CGI.

```
#!/usr/local/bin/python3
```

```
import json
```

```
import athletemodel
```

```
import yate
```

```
names = athletemodel.get_names_from_store() get_names/D_from_store()
```

```
print(yate.start_response('application/json'))
```

```
print(json.dumps(sorted(names)))
```

And here is the
"generate_data.py"
CGI.

```
#!/usr/local/bin/python3
```

```
import cgi
```

```
import json
```

```
import sys
```

```
import athletemodel
```

```
import yate
```

```
athletes = athletemodel.get_from_store()
```

```
form_data = cgi.FieldStorage()
```

```
athlete_name = form_data['which_athlete'].value
```

```
athlete = athletemodel.get_athlete_from_id(athlete_id)
```

```
print(yate.start_response('application/json'))
```

```
print(json.dumps(athletes[athlete_name].as_dict))
```

The tiniest of changes need to be made
to these CGIs, because your Android app
is only interested in your webapp's data,
NOT all of that generated HTML.

Add this code to
"yate.py" to support
the creation of radio
buttons that provide
a value for the
button that differs
from the button
text.

```
def radio_button_id(rb_name, rb_value, rb_id):  
    return('<input type="radio" name="' + rb_name +  
        '" value="' + str(rb_id) + '> ' + rb_value + '<br />')
```

A third argument lets you specify
an ID to go with the radio button.

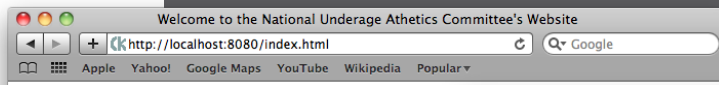


Test Drive

Before you run your amended webapp, be sure to move your SQLite database into the top-level directory of your webapp (that is, into the same folder your `index.html` file). That way, your model code can find it, so move it into your webapp's root folder *now*. When you are ready, take your SQL-powered webapp for a spin.

Start (or restart) your web server.

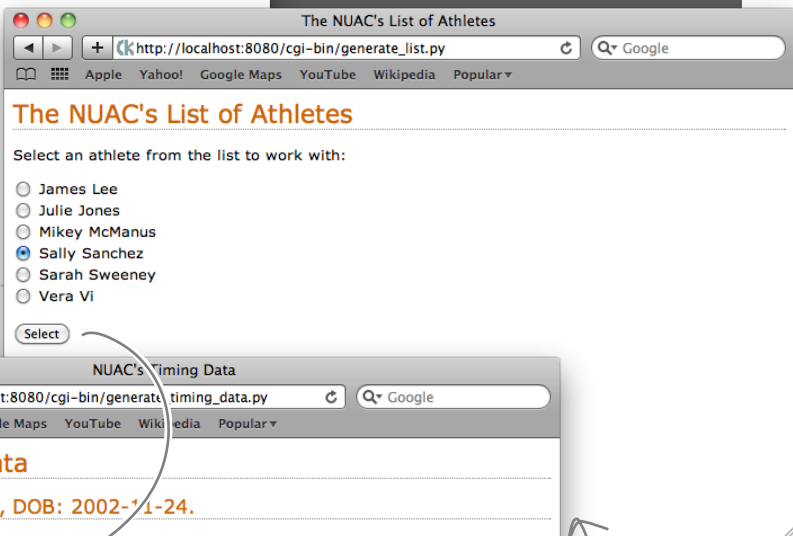
```
File Edit Window Help StartYourWebEngine
$ python3 simple_httpd.py
Starting simple_httpd on port: 8080
```



Welcome to the NUAC's Website.

Here is our athlete's **timing data**. Enjoy!

See you on the track!



The NUAC's List of Athletes

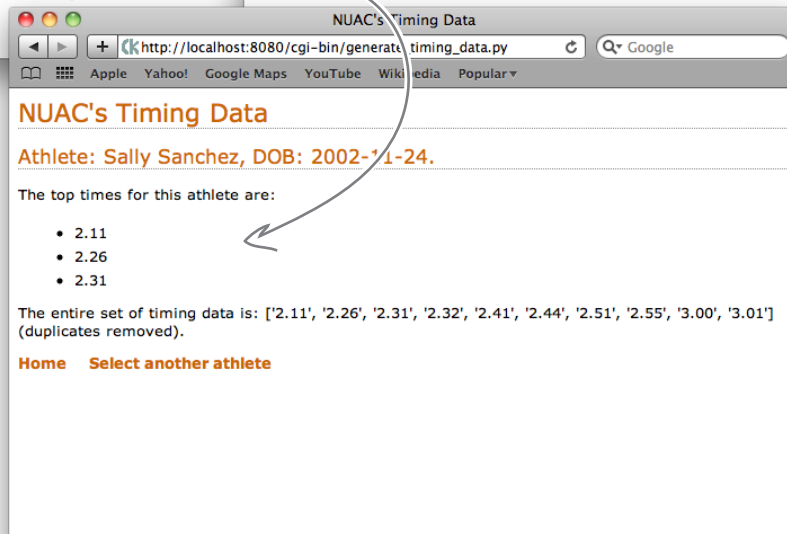
Select an athlete from the list to work with:

- ☐ James Lee
- ☐ Julie Jones
- ☐ Mikey McManus
- ☒ Sally Sanchez
- ☐ Sarah Sweeney
- ☐ Vera Vi

Select

Click on the link on the home page.

And there's Sally's timing data.



NUAC's Timing Data

Athlete: Sally Sanchez, DOB: 2002-11-24.

The top times for this athlete are:

- 2.11
- 2.26
- 2.31

The entire set of timing data is: ['2.11', '2.26', '2.31', '2.32', '2.41', '2.44', '2.51', '2.55', '3.00', '3.01'] (duplicates removed).

[Home](#) [Select another athlete](#)

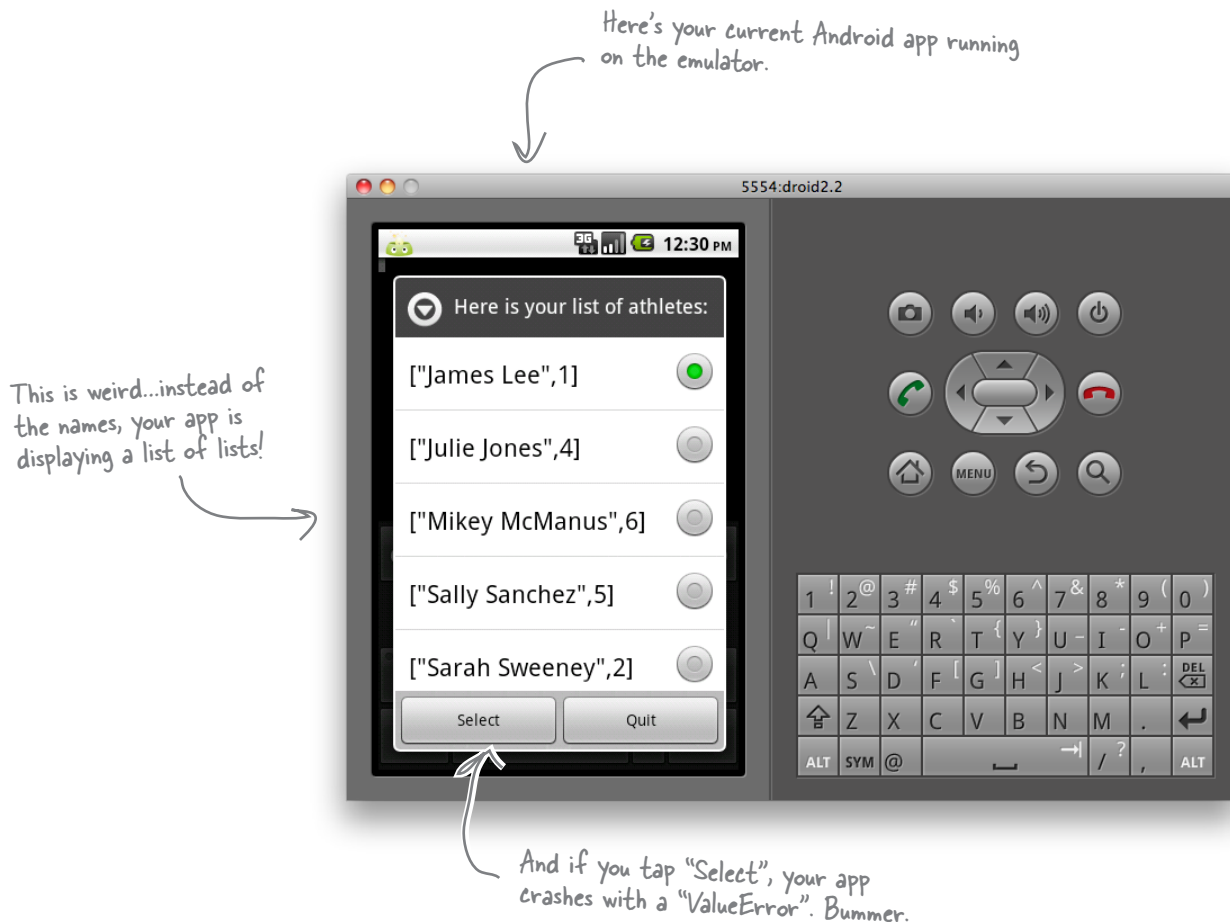
Display the list of athlete names as radio buttons.

That worked well. But what about your Android app?

You need to amend your Android app, too

Unlike your HTML-based webapp, where all of your code resides and is executed on your web server, your Android app runs on your phone and it is programmed to work with a list of names, not a list of names *and* athlete IDs.

When you run `coachapp.py` on your emulator, weirdness ensues...



Just like with the CGI scripts, you need to amend you Android app to work with the data that's now arriving from your web server—that is, a list of lists as opposed to a list.

That shouldn't take too long, should it?



Exercise

Here is your current `coachapp.py` code, which you need to amend to support the way your webapp's model now works. Grab a pencil and make the necessary changes to this code.

```
import android, json, time

from urllib import urlencode
from urllib2 import urlopen

hello_msg      = "Welcome to NUAC's Timing App"
list_title     = 'Here is your list of athletes:'
quit_msg       = "Quitting NUAC's App."
web_server     = 'http://192.168.1.34:8080'
get_names_cgi  = 'cgi-bin/generate_names.py'
get_data_cgi   = '/cgi-bin/generate_data.py'

def send_to_server(url, post_data=None):
    # There is no change to this code from the previous chapter.

app = android.Android()

def status_update(msg, how_long=2):
    # There is no change to this code from the previous chapter.

status_update(hello_msg)
athlete_names = sorted(json.loads(send_to_server(web_server + get_names_cgi)))
app.dialogCreateAlert(list_title)
app.dialogSetSingleChoiceItems(athlete_names)
app.dialogSetPositiveButtonText('Select')
app.dialogSetNegativeButtonText('Quit')
app.dialogShow()
resp = app.dialogGetResponse().result

if resp['which'] in ('positive'):
    selected_athlete = app.dialogGetSelectedItems().result[0]
    which_athlete = athlete_names[selected_athlete]
    athlete = json.loads(send_to_server(web_server + get_data_cgi, {'which_athlete': which_athlete}))
    athlete_title = athlete['Name'] + ' (' + athlete['DOB'] + '), top 3 times:'
    app.dialogCreateAlert(athlete_title)
    app.dialogSetItems(athlete['Top3'])
    app.dialogSetPositiveButtonText('OK')
    app.dialogShow()
    resp = app.dialogGetResponse().result
status_update(quit_msg)
```



Exercise Solution

Here is your current `coachapp.py` code, which you need to amend to support the way your webapp's model now works. You were to grab a pencil and make the necessary changes to this code.

```
import android, json, time
from urllib import urlencode
from urllib2 import urlopen

hello_msg      = "Welcome to NUAC's Timing App"
list_title     = 'Here is your list of athletes:'
quit_msg       = "Quitting NUAC's App."
web_server     = 'http://192.168.1.34:8080'
get_names_cgi  = 'cgi-bin/generate_names.py'
get_data_cgi   = '/cgi-bin/generate_data.py'

def send_to_server(url, post_data=None):
    # There is no change to this code from the previous chapter.

app = android.Android()

def status_update(msg, how_long=2):
    # There is no change to this code from the previous chapter.

status_update(hello_msg)

athlete_names = sorted(json.loads(send_to_server(web_server + get_names_cgi)))
athlete_names = [ath[0] for ath in athlete_names]
app.dialogCreateAlert(list_title)
app.dialogSetSingleChoiceItems(athlete_names)
app.dialogSetPositiveButtonText('Select')
app.dialogSetNegativeButtonText('Quit')
app.dialogShow()
resp = app.dialogGetResponse().result

if resp['which'] in ('positive'):
    selected_athlete = app.dialogGetSelectedItem().result[0]
    which_athlete = athlete_names[selected_athlete]
    athlete = json.loads(send_to_server(web_server + get_data_cgi, {'which_athlete': which_athlete}))
    athlete_title = athlete['Name'] + ' (' + athlete['DOB'] + '), top 3 times:'
    app.dialogCreateAlert(athlete_title)
    app.dialogSetItems(athlete['top3'])
    app.dialogSetPositiveButtonText('OK')
    app.dialogShow()
    resp = app.dialogGetResponse().result
    status_update(quit_msg)
```

Extract the athlete names *ONLY* from the list of lists.

This is a cool use of a comprehension.

Determine the ID associated with the selected athlete.

A small adjustment to next line is needed to access the "top3" attribute.

Android Pool Puzzle



Your **job** is to take the code from the pool and place it into the blank lines in the program. Your **goal** is to write the code to have your app provide the user with a mechanism to add a timing value to the server for the currently selected athlete. For now, send your data to the `cgi-bin/add_timing_data.py` CGI script.

Hint: the code from `get2inputsapp.py` (from earlier in this chapter) should come in handy here.

```
app.dialogSetNegativeButtonText('Add Time')
```

← Add another button to the existing dialog in the current version of your app.

```
...
```

```
if resp['which'] in ('positive'):
```

```
    pass
```

```
elif resp['which'] in ('negative'):
```

```
.....
.....
.....
.....
.....
.....
.....
.....
.....
```

← Based on the button that's tapped, either do nothing ("pass") or start a new dialog with the user.

```
send_to_server(web_server + add_time_cgi, {'Time': new_time, 'Athlete': which_athlete})
```

↖ If some input is supplied, send it to the web server together with the athlete's ID.



Android Pool Puzzle Solution



Your **job** was to take the code from the pool and place it into the blank lines in the program. Your **goal** was to write the code to have your app provide the user with a mechanism to add a timing value to the server for the currently selected athlete. For now, you were to send your data to the `cgi-bin/add_timing_data.py` CGI script.

Hint: the code from `get2inputsapp.py` (from earlier in this chapter) should come in handy here.

```
app.dialogSetNegativeButtonText('Add Time')
```

```
...
```

```
if resp['which'] in ('positive'):
```

```
    pass
```

```
elif resp['which'] in ('negative'):
```

```
    timing_title = 'Enter a new time'
```

```
    timing_msg = 'Provide a new timing value ' + athlete['Name'] + ': '
```

```
    add_time_cgi = '/cgi-bin/add_timing_data.py'
```

```
    resp = app.dialogGetInput(timing_title, timing_msg).result
```

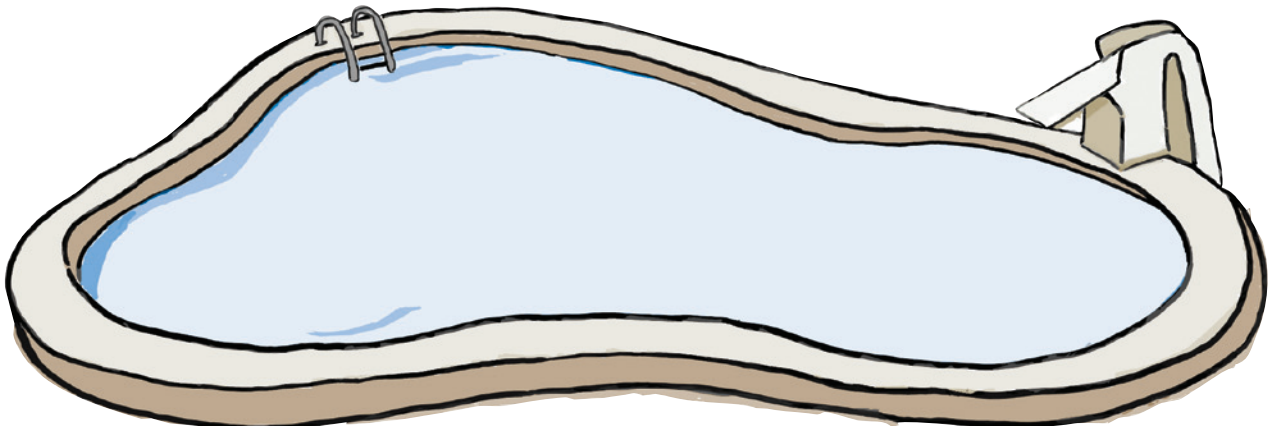
```
    if resp is not None:
```

```
        new_time = resp
```

```
        send_to_server(web_server + add_time_cgi, {'Time': new_time, 'Athlete': which_athlete})
```

Define the dialog's titles and specify the CGI to send the data to.

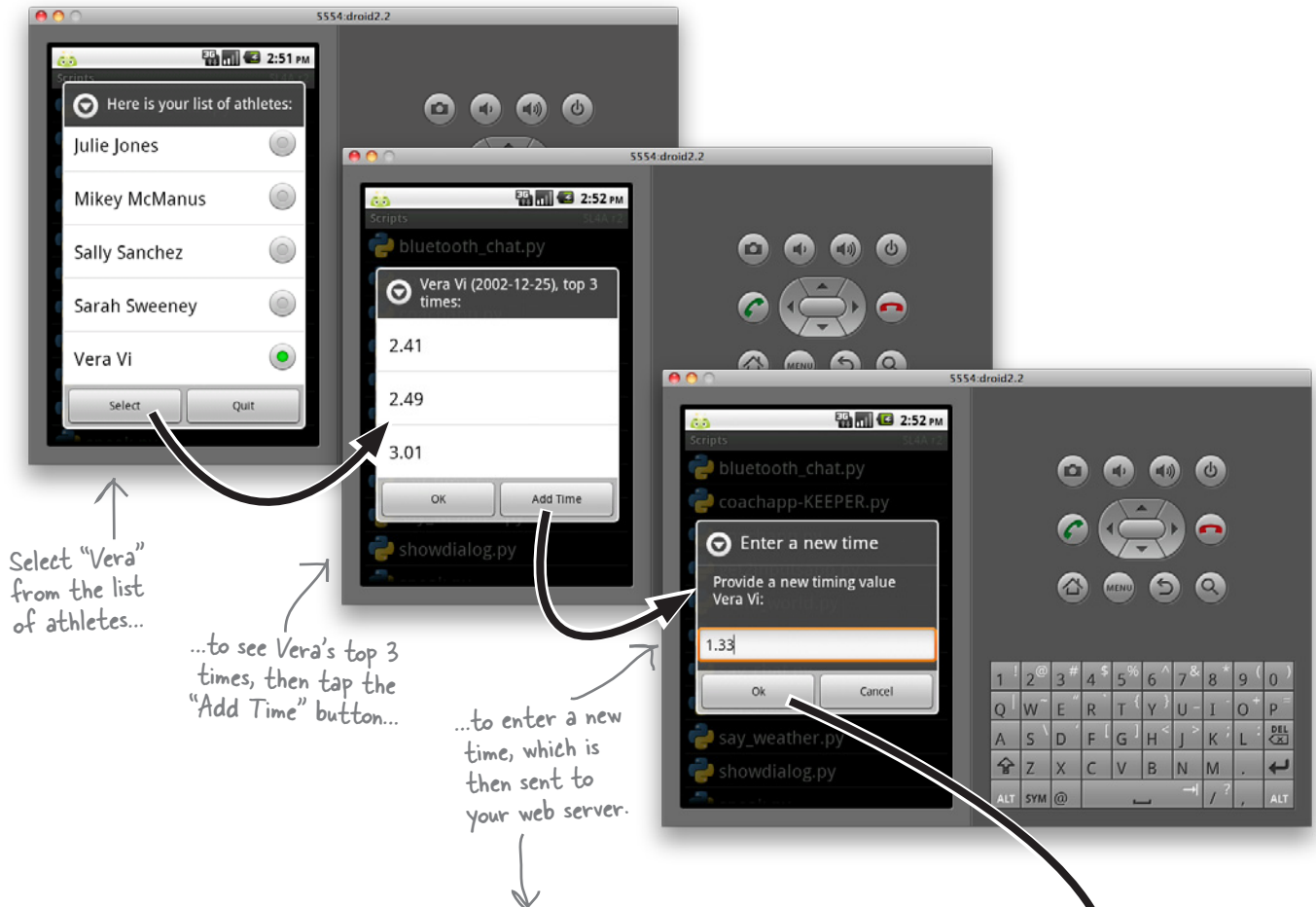
Display the dialog and wait for some user input.





Test Drive

Use the `tools/adb` command to copy your latest app to the emulator, and give your app a go.



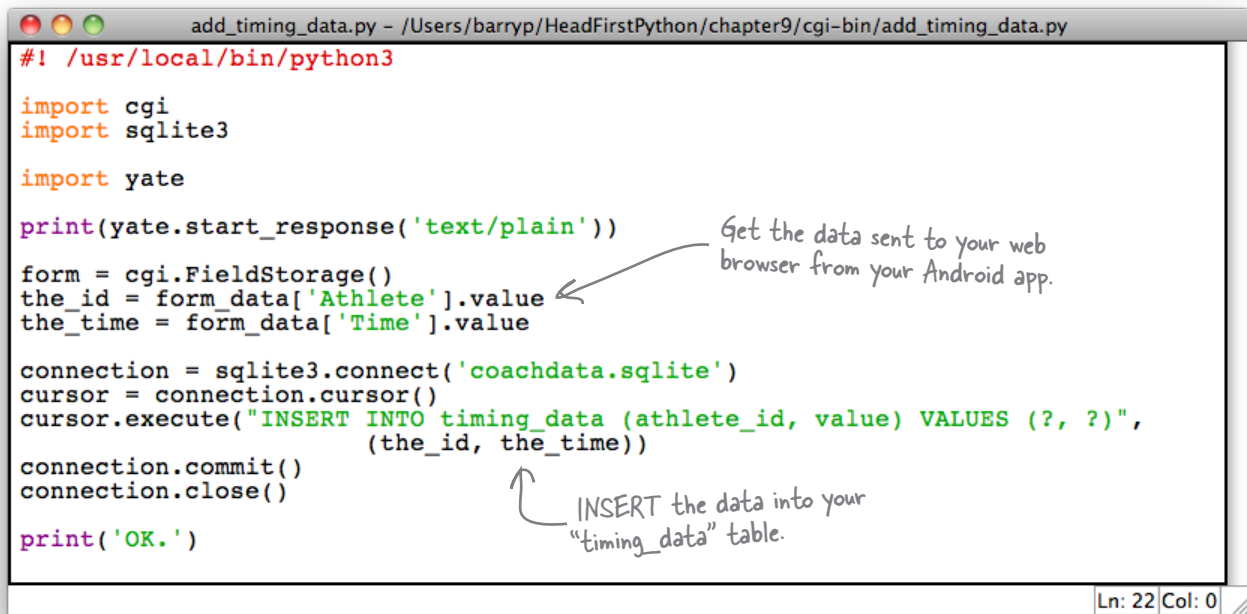
Great: your data is sent from your Android app to your web server.

```
File Edit Window Help DataFromAndroid
$ python3 simple_httpd.py
Starting simple_httpd on port: 8080
198.162.1.34 - - [27/Sep/2010 14:51:47] "GET /cgi-bin/generate_names.py HTTP/1.1" 200 -
198.162.1.34 - - [27/Sep/2010 14:52:01] "POST /cgi-bin/generate_data.py HTTP/1.1" 200 -
198.162.1.34 - - [27/Sep/2010 14:52:19] "POST /cgi-bin/add_timing_data.py HTTP/1.1" 200 -
localhost, 198.162.1.34, Mon Sep 27 14:52:19 2010: POST: Athlete->3 Time->1.33
```

Update your SQLite-based athlete data

All that's left is to change the `cgi-bin/add_timing_data.py` CGI script to write your submitted data to your database, as opposed to the web server's console screen.

At this point, it's a trivial exercise, because a single SQL `INSERT` statement will do the heavy lifting.



The screenshot shows a code editor window titled `add_timing_data.py - /Users/barryp/HeadFirstPython/chapter9/cgi-bin/add_timing_data.py`. The code is as follows:

```
#!/usr/local/bin/python3

import cgi
import sqlite3

import yate

print(yate.start_response('text/plain'))

form = cgi.FieldStorage()
the_id = form_data['Athlete'].value
the_time = form_data['Time'].value

connection = sqlite3.connect('coachdata.sqlite')
cursor = connection.cursor()
cursor.execute("INSERT INTO timing_data (athlete_id, value) VALUES (?, ?)",
               (the_id, the_time))

connection.commit()
connection.close()

print('OK.')
```

Handwritten annotations include:

- An arrow pointing from the text "Get the data sent to your web browser from your Android app." to the lines `the_id = form_data['Athlete'].value` and `the_time = form_data['Time'].value`.
- An arrow pointing from the text "INSERT the data into your 'timing_data' table." to the `cursor.execute` line.

The bottom right corner of the editor shows "Ln: 22 Col: 0".

With this version of your CGI script running on your web server, any new times entered by anyone on an Android phone are added to the data in the database.

The NUAC no longer has to worry about adding data to text files, because the files are effectively obsoleted by the use of SQLite.

You've produced a robust solution that is more manageable, scalable, programmable, and extendable. And it's all thanks to the power of Python, its database API and the inclusion of `sqlite3` in the standard library.

All that's left to do is sit back, relax and bask in the glory of your latest programming creation...

The NUAC is over the moon!

Of course, your use of SQLite gives you more than just easy insertions of data. With the NUAC's data in tables, it's easy to answer some of the questions that have been on their mind.



To answer these and other queries on the data in the NUAC's database, you'll have to bone up on your SQL. Then it's up to you to take it from there.

You've converted your webapp to use an SQL database. As your data management needs increase, you can consider alternative *heavy-duty* data management technologies as needed.

This is great work. Your webapp is ready for the big time.



Your Python Toolbox

You've got Chapter 9 under your belt and you've added some key Python tools to your every expanding Python toolbox.

Python Lingo

- "Database API" – a standardized mechanism for accessing an SQL-based database system from within a Python program.

Database Lingo

- "Database" – a collection of one or more tables.
- "Table" – a collection of one or more rows or data, arranged as one or more columns.
- "SQL" – the "Structured Query Language" is the language of the database world and it lets you work with your data in your database using statements such as `CREATE`, `INSERT`, and `SELECT`.



BULLET POINTS

- The `fieldStorage()` method from the standard library's `cgi` module lets you access data sent to your web server from within your CGI script.
- The standard `os` library includes the `environ` dictionary providing convenient access to your program's environment settings.
- The SQLite database system is included within Python as the `sqlite3` standard library.
- The `connect()` method establishes a connection to your database file.
- The `cursor()` method lets you communicate with your database via an existing connection.
- The `execute()` method lets you send an SQL query to your database via an existing cursor.
- The `commit()` method makes changes to your database **permanent**.
- The `rollback()` method **cancels** any pending changes to your data.
- The `close()` method closes an existing connection to your database.
- The "?" placeholder lets you parameterize SQL statements within your Python code.